

0. 简历术语速背卡

《低延迟语音对话闭环》

- 一句话定义：像点单到上菜尽量快。
- 在本项目里指什么：用户说话后，系统完成识别、思考、合成和播放。
- 面试可直接背诵版（口播 30~60 字）：我做的是一条完整语音对话链路，目标是让用户说完话后，系统能尽快识别、生成回答并播出来。

《VOICE / LLM / TTS 三模块》

- 一句话定义：像餐厅前台、厨师、上菜员。
- 在本项目里指什么：VOICE 负责听和控轮次，LLM 负责生成文本，TTS 负责合成播放。
- 面试可直接背诵版（口播 30~60 字）：我把系统拆成 VOICE、LLM、TTS 三块，分别处理语音输入、文本生成和语音播放，边界比较清楚。

《ASR》

- 一句话定义：把人说的话写成字。
- 在本项目里指什么：VOICE 模块里负责把麦克风语音转成文本。
- 面试可直接背诵版（口播 30~60 字）：ASR 在我的项目里就是语音识别，把用户说的话转成文字，再交给 LLM 去生成回复。

《端侧 / 云端》

- 一句话定义：端侧像家里，云端像远程厨房。
- 在本项目里指什么：ASR 和 TTS 在本机跑，LLM 走云端接口。
- 面试可直接背诵版（口播 30~60 字）：我把实时采集和播放放在端侧，文本生成放在云端，这样本地负责交互节奏，云端负责回答质量。

《ZeroMQ 混合通信模型》

- 一句话定义：不同路段用不同送信方式。
- 在本项目里指什么：握手反馈用 REQ/REP，文本分句推送用 PUSH/PULL。
- 面试可直接背诵版（口播 30~60 字）：我没有把所有通信都做成一种模式，而是按语义拆开，控制类消息一问一答，流式文本单向连续推送。

《REQ/REP》

- 一句话定义：像对讲机一问一答。
- 在本项目里指什么：VOICE 和 LLM、VOICE 和 TTS 之间做严格交替通信。
- 面试可直接背诵版（口播 30~60 字）：REQ/REP 我用在必须有确认的地方，比如一轮开始和播放结束，它的特点是一问一答、严格交替。

《PUSH/PULL》

- 一句话定义：像传送带连续送餐。
- 在本项目里指什么：LLM 把分句文本持续推给 TTS。
- 面试可直接背诵版（口播 30~60 字）：PUSH/PULL 我用在 LLM 到 TTS，LLM 只管把句子片段往下发，TTS 只管接收处理。

《一轮对话握手》

- 一句话定义：像进门先确认点单。
- 在本项目里指什么：VOICE 把一轮 ASR 文本交给 LLM，并收到确认。
- 面试可直接背诵版（口播 30~60 字）：一轮对话握手就是用户说完一句后，VOICE 把文本发给 LLM，LLM 先确认收到，避免前端一直卡住。

《播放结束反馈》

- 一句话定义：像上菜员说这桌上完了。
- 在本项目里指什么：TTS 播完最后一段后通知 VOICE 开新一轮。
- 面试可直接背诵版（口播 30~60 字）：播放结束反馈是 TTS 告诉 VOICE，这一轮回答已经播完，VOICE 再开始下一轮采集，避免回声被识别进去。

《流式分句单向推送》

- 一句话定义：边切小段边往前送。
- 在本项目里指什么：LLM 边收到回复边切句，再连续推给 TTS。
- 面试可直接背诵版（口播 30~60 字）：LLM 不等整段回答生成完，而是边收边按句子切开，然后单向推给 TTS，让播放准备更早开始。

《云端流式对话 API》

- 一句话定义：像电话里边说边听。
- 在本项目里指什么：LLM 模块从云端持续接收回复增量。
- 面试可直接背诵版（口播 30~60 字）：我接的是云端流式对话接口，它会持续返回文本增量，我可以在回答没完整结束时就开始处理。

《增量回调》

- 一句话定义：来一小段就提醒一次。
- 在本项目里指什么：每收到一段 LLM 输出就触发切句处理。
- 面试可直接背诵版（口播 30~60 字）：增量回调就是云端每吐出一内容，我的代码就立刻拿到这一小段，接着判断能不能组成可播放句子。

《中文强弱标点切句策略》

- 一句话定义：像按逗号句号分餐盘。

- 在本项目里指什么：按 。 ！ ？ ； 和 ， 、 ： 等标点决定切句时机。
- 面试可直接背诵版（口播 30~60 字）：我用中文强弱标点做切句，强标点更适合直接成句，弱标点要结合长度阈值，避免句子太碎。

《句子片段实时下发》

- 一句话定义：做出一盘先上一盘。
- 在本项目里指什么：LLM 切出可读片段后马上发给 TTS。
- 面试可直接背诵版（口播 30~60 字）：只要 LLM 输出形成一个适合朗读的句子片段，我就马上下发给 TTS，不等完整回答结束。

《token》

- 一句话定义：像一句话里的小积木。
- 在本项目里指什么：云端回复时逐步返回的文本小单元。
- 面试可直接背诵版（口播 30~60 字）：token 可以理解成模型输出的文本小颗粒，我会根据这些增量内容逐步拼接，再判断是否能切成句子。

《文本队列 / 音频队列》

- 一句话定义：像两个排队窗口。
- 在本项目里指什么：文本先排队合成，音频再排队播放。
- 面试可直接背诵版（口播 30~60 字）：TTS 里我设计了文本队列和音频队列，文本进入合成流程，合成后的音频再进入播放流程。

《双线程异步模型》

- 一句话定义：像一人做饭一人上菜。
- 在本项目里指什么：一个线程合成音频，一个线程播放音频。
- 面试可直接背诵版（口播 30~60 字）：TTS 里合成和播放分开跑，合成线程负责生产音频，播放线程负责消费音频，两边互不长时间卡住。

《流式切句前置投递》

- 一句话定义：还没全做完先送前菜。
- 在本项目里指什么：LLM 首句成形后立即推给 TTS。
- 面试可直接背诵版（口播 30~60 字）：我把切句和投递放在流式接收过程中，首句一旦满足朗读条件，就直接推给 TTS 准备发声。

《前移首句发声时机》

- 一句话定义：让第一口菜更早上桌。
- 在本项目里指什么：首句不等完整回答，提前合成播放。
- 面试可直接背诵版（口播 30~60 字）：我的优化重点是首句发声时机，LLM 生成到可读首句后马上下发，让用户更快听到系统回应。

《整轮语音交互闭环 4 s 以内》

- 一句话定义：从说话到听答控制很快。
- 在本项目里指什么：识别、生成、合成、播放的一轮控制在 4 秒内。
- 面试可直接背诵版（口播 30~60 字）：最终效果是一轮语音交互从用户说完到系统完成回答播放，整体闭环可以控制在 4 秒以内。

A. 项目自我介绍口播稿

我做的是一个基于 C++ 的低延迟语音交互系统。我把系统分成 VOICE、LLM、TTS 三个独立模块，VOICE 负责麦克风采集、端侧 ASR 和轮次控制，LLM 负责生成文本回复，TTS 负责端侧语音合成和播放。

整体链路是用户说话后，端侧 ASR 先把语音转成文本，再交给云端 LLM 做流式生成，最后端侧 TTS 把回复合成语音播出来。这样实时采集和播放都留在本地，生成能力交给云端，整体职责比较清晰。

通信上我用了 ZeroMQ 的混合模型。VOICE 和 LLM 之间用 REQ/REP 做一轮对话握手，VOICE 和 TTS 之间也用 REQ/REP 接收播放结束反馈；LLM 到 TTS 用 PUSH/PULL，因为这里更像文本传送带，LLM 按句子片段连续往下发，不需要等 TTS 回应。

LLM 侧我接入云端流式对话接口，通过增量回调拿到持续输出，再用中文强弱标点切句策略，把可朗读的句子片段实时下发给 TTS。TTS 侧我设计了文本队列和音频队列，合成线程负责生产音频，播放线程负责消费音频。通过流式切句和双队列异步处理，首句发声时机明显靠前，整轮语音交互闭环可以控制在 4 秒以内。

B. 面试问答清单

Q1. (项目背景与架构整体设计) 你这个项目主要解决什么问题？

考察意图：面试官想看你能不能用业务语言讲清楚项目价值，而不是只堆技术点。

回答要点：

- 我做的是一个低延迟语音交互系统，核心目标是让用户说完话后，系统能尽快识别、生成回答并播出来。
- 整条链路包含 ASR、LLM、TTS，也就是语音转文字、生成回复、文字转语音三个步骤。
- 我最关注的是首句发声速度和整轮闭环时间，所以设计上尽量让各模块并行工作。
- 最终一轮语音交互可以控制在 4 秒以内，听感上不会有明显长时间空等。

Q2. (项目背景与架构整体设计) 为什么拆成 VOICE、LLM、TTS 三个模块？

考察意图：面试官想看你对模块边界、职责拆分和进程间协作是否有清晰理解。

回答要点：

- 我把 VOICE 作为入口和轮次控制模块，负责采集、ASR 触发和下一轮开始时机。
- LLM 只负责文本生成和流式切句，避免把音频播放这类实时逻辑混进去。
- TTS 专注合成和播放，这样文本生成慢一点时不会直接影响音频播放流程。
- 我考虑过把所有逻辑写在一个进程里，但这样调试和故障隔离都更麻烦，所以我选择拆成三个独立模块。

Q3. (项目背景与架构整体设计) 端侧和云端分别放了哪些能力？

考察意图：面试官想确认你是否理解本地实时性和云端生成能力之间的取舍。

回答要点：

- 端侧放了 ASR、麦克风采集、TTS 合成和播放，这些都和实时交互体验直接相关。
- 云端负责 LLM 文本生成，因为生成质量和模型能力更依赖云端能力。
- 我这样做的考虑是，本地控制用户听说节奏，云端负责回答内容。
- 另一种做法是把更多能力都放到本地，但模型能力和部署成本会更受限制，所以我没有这么选。

Q4. (ZeroMQ 基础与本项目通信模式选型) ZMQ 是什么？你在项目里怎么用的？

考察意图：面试官想看你能不能用通俗语言解释通信组件，以及是否真正落到项目场景。

回答要点：

- 我理解 ZMQ 就是进程之间传消息的一套通信工具，可以把不同模块连接起来。
- 在我的项目里，VOICE、LLM、TTS 是三个独立进程，它们之间通过 ZMQ 交换文本和控制信号。
- 我主要用了两类模式：REQ/REP 做一问一答，PUSH/PULL 做连续单向推送。
- 这样每条通信链路的语义比较明确，控制消息和流式文本不会混在一起。

Q5. (ZeroMQ 基础与本项目通信模式选型) REQ/REP 是怎么工作的 ?

考察意图：面试官想看你是否理解一问一答、严格交替这个核心特点。

回答要点：

- REQ/REP 可以理解成对讲机式的一问一答，请求方发一句，响应方必须回一句。
- 它要求严格交替，所以适合有明确确认关系的地方。
- 我在 VOICE 到 LLM 用它做一轮对话握手，LLM 收到 ASR 文本后先回确认。
- 我在 VOICE 到 TTS 也用它，TTS 播完最后一句后回复 VOICE，VOICE 再进入下一轮采集。

Q6. (ZeroMQ 基础与本项目通信模式选型) 为什么不全部用 REQ/REP ?

考察意图：面试官想看你是否知道通信模式和业务语义要匹配。

回答要点：

- REQ/REP 的优点是确认关系清楚，但它必须一问一答、严格交替。
- LLM 到 TTS 这条链路是持续产生句子片段，更像上游不断发、下游不断收。
- 如果这里也用 REQ/REP，LLM 每发一段都要等回应，会拖慢流式投递节奏。
- 所以我把控制类链路用 REQ/REP，把连续文本链路用 PUSH/PULL。

Q7. (ZeroMQ 基础与本项目通信模式选型) 为什么不用发布订阅模式 ?

考察意图：面试官想看你对模式选型是否有边界意识，而不是只会套模板。

回答要点：

- 我考虑过发布订阅，但这个项目里 LLM 到 TTS 是明确的一路下发，不需要广播给多个订阅方。
- PUSH/PULL 更符合我的场景，上游只管发、下游只管收，也天然适合连续任务投递。
- 发布订阅更适合一份消息给多个接收方，而我的 TTS 消费语义更像队列。
- 所以我没有选发布订阅，而是让 LLM 到 TTS 保持单向推送关系。

Q8. (云端 LLM 流式接入) 增量回调在你的 LLM 模块里怎么工作 ?

考察意图：面试官想看你是否理解流式接收，以及它如何服务低延迟。

回答要点：

- 云端生成回复时不是一次性返回完整文本，而是持续返回文本增量。
- 我的 LLM 模块每收到一小段增量，就进入回调处理逻辑。
- 回调里会把增量拼到缓冲区，再交给中文标点切句器判断能不能切出可朗读片段。
- 一旦能切句，我就马上推给 TTS，不等完整回答结束。

Q9. (云端 LLM 流式接入) 半句话被拆开返回时怎么处理 ?

考察意图：面试官想看你是否处理了流式输出中的边界问题。

回答要点：

- 流式输出经常不是完整句子，可能只返回半句话或者几个字。
- 我不会直接把每个增量都发给 TTS，而是先放进文本缓冲区。
- 缓冲区会结合强弱标点、最小字数和最大句长来判断切句时机。
- 我踩过坑的一个坑是切得太碎会导致播音不自然，所以弱标点必须配合长度阈值。

Q10.（云端 LLM 流式接入）你怎么做超时和取消机制？

考察意图：面试官想看你是否考虑了异常链路，而不是只覆盖正常流程。

回答要点：

- 我做了连接超时、读空闲超时和单轮总超时三层控制，分别覆盖不同卡住场景。
- 如果流式接收一直没有有效输出，我会做一次非流式回退，避免整轮完全无响应。
- 新一轮请求到来时，我会触发当前请求取消，防止多个轮次的回复混在一起。
- 这里我没有只用一个总超时，因为定位问题时不够细，也不利于区分连接慢还是生成慢。

Q11.（中文流式切句策略与首音延迟优化）你的中文强弱标点切句是怎么设计的？

考察意图：面试官想看你是否针对中文语音播放做了细粒度处理。

回答要点：

- 我把句号、问号、感叹号、分号、换行看成强标点，它们通常适合作为一句结束。
- 逗号、顿号、冒号属于弱标点，我会结合句子长度再决定要不要切。
- 同时我设置了首句最小字数、普通句最小字数和最大句长，避免太短或太长。
- 这样既能尽早下发首句，也能保证 TTS 播出来不至于太零碎。

Q12.（中文流式切句策略与首音延迟优化）为什么不直接按 token 数切？

考察意图：面试官想看你是否理解文本切分和自然朗读之间的关系。

回答要点：

- 我考虑过按 token 数切，但这种方式不理解中文语义停顿。
- 固定数量切开可能刚好切在词语中间，TTS 播起来会很生硬。
- 中文标点更接近日常说话的停顿位置，所以我优先按强弱标点切句。
- token 数在我这里更适合作为辅助观察，不适合作为唯一切句标准。

Q13.（中文流式切句策略与首音延迟优化）你怎么优化首音延迟？

考察意图：面试官想看你是否能把低延迟目标拆到具体链路上。

回答要点：

- 我没有等 LLM 完整回答结束，而是在流式接收过程中就开始切句。
- 首句只要达到最小字数并遇到合适标点，就立即投递给 TTS。

- TTS 收到文本后进入合成队列，播放线程可以消费已经合成好的音频。
- 这里的关键是让 LLM 生成、TTS 合成和播放准备重叠起来，而不是串成一条长等待链。

Q14. (TTS 双队列生产者-消费者并发模型) TTS 为什么要设计文本队列和音频队列？

考察意图：面试官想看你是否理解并发解耦，而不是简单堆线程。

回答要点：

- 文本队列接收 LLM 推来的句子片段，音频队列保存已经合成好的音频。
- 合成线程从文本队列取任务，生成音频后放进音频队列。
- 播放线程只关心音频队列，有音频就播放，没有就阻塞等待。
- 我考虑过一个线程串行做合成和播放，但这样合成时不能播放，播放时也不能提前合成下一句。

Q15. (TTS 双队列生产者-消费者并发模型) 生产者-消费者模型你怎么口语化理解？

考察意图：面试官想看你能否把并发模型讲清楚，且不依赖底层调用名。

回答要点：

- 我把它理解成厨房和传菜口，合成线程是厨师，播放线程是上菜员。
- 厨师做好一份音频就放到音频队列，上菜员按顺序取出来播放。
- 队列为空时，上菜员不会空转，而是阻塞等待；有新音频时再被唤醒。
- 队列访问时我用互斥保护，保证两个线程不会同时把队列状态改乱。

Q16. (TTS 双队列生产者-消费者并发模型) END 哨兵和 is_last 是怎么配合的？

考察意图：面试官想看你是否处理了轮次边界和播放结束反馈。

回答要点：

- 每一轮 LLM 输出结束时，会发送 END 作为结束哨兵。
- TTS 的文本队列根据 END 识别这一轮文本已经收完。
- 合成到最后一段音频时，会在音频队列里标记 is_last。
- 播放线程播完 is_last 对应的音频后，再通过一问一答通道通知 VOICE 开始下一轮。

Q17. (健壮性、可观测性与扩展性) 如果某个模块崩溃或者卡住，你怎么定位？

考察意图：面试官想看您是否有工程化排障意识。

回答要点：

- 我把三个模块拆成独立进程，出问题时可以先看是哪条链路没有继续往下走。
- 日志里会记录轮次开始、LLM 首次输出、切句投递、TTS 播放结束这些关键节点。
- 超时也分了连接、读空闲和单轮总超时，能帮助我判断卡在连接、生成还是整体流程。
- 敏感信息会做脱敏，避免排查日志时把密钥直接打出来。

Q18. (健壮性、可观测性与扩展性) 怎么避免 TTS 播放时被 ASR 当成用户说话？

考察意图：面试官想看你是否理解语音交互里的回声误识别问题。

回答要点：

- 我让 VOICE 通过 TTS 的播放结束反馈来控制下一轮采集时机。
- TTS 播放过程中，VOICE 不急着开启新一轮识别，避免把系统自己的声音识别成用户输入。
- 播放线程播完最后一段音频后，才通知 VOICE 进入下一轮。
- 这里我没有只靠音量阈值判断，因为环境噪声和扬声器回声会让判断不稳定。

Q19. (健壮性、可观测性与扩展性) API Key 和模型配置你怎么管理？

考察意图：面试官想看你是否具备基本安全意识和配置治理能力。

回答要点：

- API Key 我只通过环境变量加载，不写进配置文件和代码里。
- 日志输出时会做脱敏，避免排障时泄露敏感信息。
- 模型名会做白名单校验，防止配置填错导致请求跑到不可控模型上。
- 普通参数走配置文件，敏感参数走环境变量，这样本地调试和部署都比较清晰。

Q20. (健壮性、可观测性与扩展性) 如果要支持多轮历史，你会怎么扩展？

考察意图：面试官想看你的设计是否留了扩展空间。

回答要点：

- LLM 请求结构里我已经按消息数组组织，默认包含系统提示和用户输入。
- 如果要加多轮历史，可以在数组中追加轮次上下文，并控制总长度。
- 我会优先保留最近几轮和必要摘要，避免历史过长拖慢生成。
- 这样扩展主要集中在 LLM 模块，不需要大改 VOICE 和 TTS 的通信边界。